

Воркшоп:

Спец-Driven Development (SDD) для AI агентов

От теории к практике: Создай спес для своей фичи

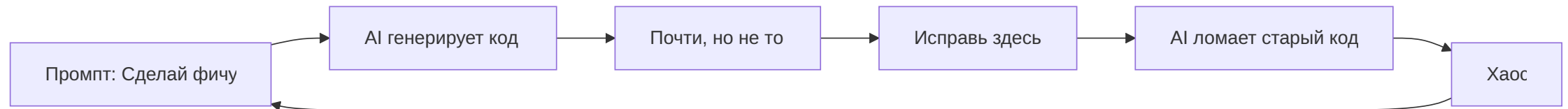
Цели :

- Освоить 6 этапов SDD на практике
- Написать полную спес.md для реальной задачи
- Протестировать на AI-агенте (домашка)
- **Результат: Готовая спес в твоём репозитории**

Icebreaker

Твой опыт вайб-кодинга?

- "Сколько раз AI сломал код за сессию?"
- **Проблема:** Цикл хаоса без spec



Что такое SDD?

- **SDD:** Спецификация — первична, код генерируется из неё
- Спецификация = "живой" артефакт: поведение, интерфейсы, edge cases
- **Принцип:** Сначала опиши → AI строит
- Роль dev: Architect of Context (не coder)

Инструменты

Контекстная инженерия

- Проблема шума: Только релевантные файлы в контекст.
- Используй `AGENTS.md` + спес как якорь.

Инструменты

Фреймворк	Установка	Интеграция	Автоматизация	Кривая обучения	Лучш
GitHub Spec Kit	<code>uv tool install</code>	GitHub, VS Code, Cursor	Высокая (CLI commands)	Средняя	Командный workflow
OpenSpec	Легкая (Markdown)	Любой AI (Claude, ChatGPT)	Низкая (manual)	Низкая	Индивидуальная разработка прост
Cursor Rules	Встроено в Cursor	Только Cursor	Средняя	Низкая	Cursor разработка

Плюсы и минусы

GitHub Spec Kit

Плюсы:

- Автоматическая генерация шаблонов, диаграмм, задач
- Поддержка multiple AI agents (Claude, Copilot, Cursor)
- Структурированный workflow: constitution → specify → plan → tasks → implement
- Результат: готовый проект с тестами за минуты

Минусы:

- Кривая обучения выше (CLI commands)
- Привязка к GitHub (хотя и опциональная)
- Overhead для микро-проектов

OpenSpec

Плюсы:

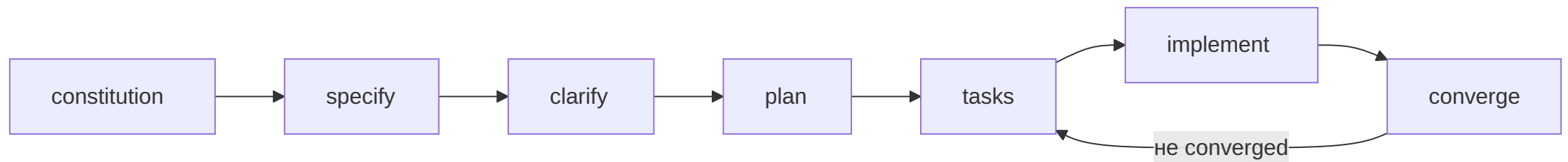
- **Минималистичный** — просто Markdown, любой AI
- Работает с любым LLM (Claude, ChatGPT, Gemini)
- Идеален для быстрого прототипирования
- Низкий порог входа для новичков

Минусы:

- Ручная работа (нет автоматизации)
- Нет встроенной интеграции с GitHub/IDE
- Требуется дисциплины в структурировании
- Сложнее масштабировать на большие команды

Что такое **GitHub Spec Kit**?

- **Toolkit для Spec-Driven Development (SDD):** превращает идеи в структурированные specs для AI-агентов (Claude, Cursor, Copilot).
- **Цель:** Заменить vibe-coding на предсказуемый процесс: /constitution → /specify → /plan → /tasks → /implement.
- **Преимущества:**
 - Генерирует шаблоны, диаграммы, задачи автоматически.
 - Интеграция с GitHub, VS Code, Cursor.
 - Результат: готовый проект с тестами за минуты.



Требования перед установкой

- **uv** — рекомендуемый менеджер пакетов (или `pipx`).
 - Проверь: `uv --version`.
- **Python 3.11+.**
- **AI-агент:** Claude Code, Cursor, Copilot в VS Code и 30+ других[3][4].
- **Опционально:** GitHub Token (GH_TOKEN) для API[5].

Подготовка: Перезапусти терминал после `uv[1]`.

Установка (актуальная версия — 1.0.1)

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v1.0.1
```

- **Проверь:** `specify version` или `specify check` [5].
- **Обновление/апгрейд** (самоменеджмент CLI):

```
specify self check                # есть ли новая версия (read-only)
specify self upgrade --dry-run    # предпросмотр без установки
specify self upgrade              # обновить до последнего релиза
specify self upgrade --tag v1.0.1 # закрепить конкретный тег
```

Для существующих проектов: `specify init . --integration claude --`
`here` [5].

Инициализация проекта

```
specify init myproject --integration claude
```

- Или в текущей папке: `specify init . --force` [5].
- **Флаги:**

Флаг	Описание
<code>--integration claude</code>	Выбор агента (<code>--integration copilot</code> , <code>cursor</code> , ...)
<code>--integration-options="--skills"</code>	Установить как agent skills вместо слэш-команд
<code>--here</code>	Текущая директория

Результат: constitution.md, spec.md, .cursor/rules + шаблоны.

Первые команды в AI-агенте

Открой проект в Cursor/Claude: `/` покажет команды.

1. `/speckit.constitution` – принципы проекта
2. `/speckit.specify` – User Stories + API
3. `/speckit.clarify` – уточнение неясного
4. `/speckit.plan` – план файлов
5. `/speckit.tasks` – 10-20 задач
6. `/speckit.analyze` – проверка согласованности спеки/плана/задач
7. `/speckit.implement` – код по задачам
8. `/speckit.converge` – сверка кода со спекой, остаток → новые задачи
9. `/speckit.taskstoissues` – задачи → GitHub issues
10. `/speckit.checklist` – кастомные чек-листы качества

Экосистема: Extensions, Presets, Bundles

Спец Kit можно кастомизировать — это главная новая возможность, которой нет в базовом workflow.

Компонент	Что делает	Команда
Extension	Добавляет новые команды/возможности (Jira, ревью кода, bug-fix, оценка идей)	<code>specify</code> <code>extension add</code> <code><name></code>
Preset	Меняет формат спеки/планов/задач (Agile, Waterfall, локализация, compliance)	<code>specify preset</code> <code>add <name></code>

Экосистема: Extensions, Presets, Bundles

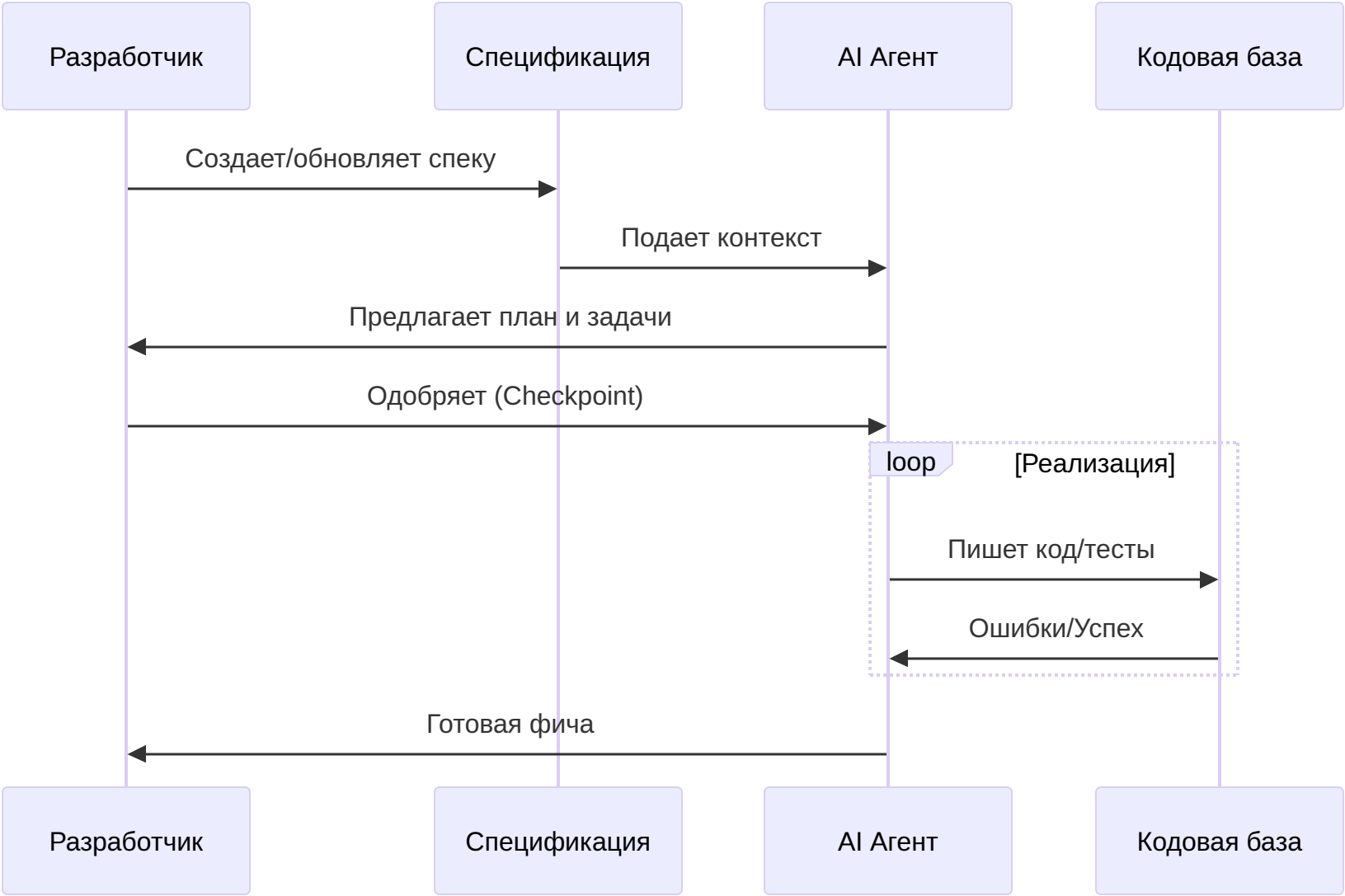
Спец Kit можно кастомизировать — это главная новая возможность, которой нет в базовом workflow.

Компонент	Что делает	Команда
Bundle	Пакет из extensions+presets под роль (PM, security, developer) — одной командой	<code>specify bundle</code> <code>install <id></code>
Workflow	Автоматизация повторяющихся шагов	<code>specify</code> <code>workflow run</code> <code><name></code>

7 Этапов SDD: Практика на твоей фиче

Выбери свою мини-фичу сейчас (2 мин): e.g. "Логин по email", "CRUD для задач", "Email-уведомление".

Визуализация процесса SDD



Этап 1 - Constitution

Что: "ДНК" проекта (стандарты, стек).

Шаблон (скопируй в `constitution.md`):

```
## Принципы разработки
1. Clean Architecture - четкое разделение слоев
2. Type Safety - TypeScript на клиенте и Python3.11 сервере
3. REST + WebSocket - REST API + real-time
4. TDD (Test-Driven Development) — разработка через тестирование:
   - _Red_: Написание провального теста перед кодом.
   - _Green_: Написание минимально необходимого кода для прохождения теста.
   - _Refactor_: Улучшение кода при сохранении работоспособности тестов.
5. Database First - схема БД определяет API
```

Этап 2 - Specify

Что: Markdown spec с User Stories, API, БД.

Шаблон `myfeature_spec.md` :

```
# Specify: [Название фичи]

## User Stories
- Как [user], я хочу [действие] чтобы [польза]

## API
- POST /api/[endpoint] {schema}

## БД
- Таблица [name]: поля...

## Ограничения
- Edge cases: [список]
```

Задание: Напиши 2-3 stories для своей фичи.

Этап 3 - Clarify

Что: AI + human аудит на противоречия/edge cases.

Задание: Скинь спес в AI, обнови по ответу. Чекпоинт: Покажи соседу.

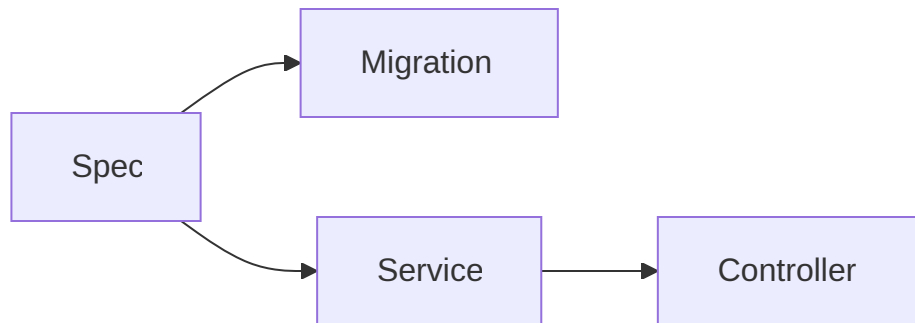
Этап 4 - Plan

Что: Высокоуровневый план файлов/зависимостей.

Шаблон в spec.md:

```
## Plan
1. [Файл1: описание]
2. [Файл2: зависимости]
```

Задание: AI-промпт: "Создай Plan для spec." Одобри/отредактируй.



Этап 5 - Tasks

Что: 10-20 атомарных задач (todo-лист).

Шаблон:

```
## Tasks
- [ ] Task1: [file] + tests
- [ ] Task2: ...
```

Задание: Разбей Plan на 10 задач. Стандарт: testable, <15 мин каждая.

Этап 6 - Implement

Что: AI выполняет по задачам, с тестами/checkpoints.

Процесс:

1. AI: "Implement Task1 из spec."
2. Run tests/linter.
3. Commit only if green.

Задание: Выбери 1 task, сгенерируй код в AI. Не коммить — просто проверь.

Этап 7 - Converge

Что: Сверка кода со спекой/планом/задачами.

- AI оценивает: всё ли реализовано, нет ли расхождений с спес.
- Несделанное/сломанное → **добавляется как новые задачи** (возврат к этапу 5).
- Повторяй implement → converge, пока `/speckit.converge` не отчитается **Converged**.

Задание: Запусти `/speckit.converge` по своей фиче. Если выдал новые задачи — обнови спес и закрой их.

Контрольные точки SDD

- Одобри Plan перед кодом
- Human review критических частей
- Формула контекста: $C_{total} = C_{sys} + C_{spec} + C_{code}$ (минимум history!)

Чекпоинт: Твоя spec готова на 80%? Групповой фидбек.

А теперь сами в командах

1. Доработай `спес` своей фичи.
2. Скинь в AI **без `спес`** → посчитай ошибки.
3. Скинь **с `спес`** → сравни.
4. Создай PR в свой репозиторий.

Q&A

Спасибо! Готовы к SDD?